

SKILL : WEEK 9

SESSION 17:

Task - 1:

To implement how to list active jobs, display status information, retrieve job IDs, format output, update state information, and test job listings.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 3

struct Job
{
    int id;
    pid_t pid;
    char state[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void show_jobs()
{
    printf("\nActive Jobs:\n");
    printf("ID\tPID\tSTATE\n");

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].pid != 0)
        {
            printf("[%d]\t%d\t%s\n",
                    jobs[i].id,
                    jobs[i].pid,
                    jobs[i].state);
        }
    }
}

int main()
{
    for (int i = 0; i < MAX_JOBS; i++)
    {
        pid_t pid = fork();

        if (pid == 0)
        {
            sleep(3 + i);
            return 0;
        }

        jobs[job_count].id = job_count + 1;
        jobs[job_count].pid = pid;
    }
}
```

```

        sprintf(jobs[job_count].state, "Running");
        job_count++;
    }

    printf("Three background jobs created.\n");
    show_jobs();

    sleep(5);

    for (int i = 0; i < job_count; i++)
    {
        if (waitpid(jobs[i].pid, NULL, WNOHANG) > 0)
        {
            sprintf(jobs[i].state, "Completed");
        }
    }

    printf("\nAfter status update:\n");
    show_jobs();

    for (int i = 0; i < job_count; i++)
        waitpid(jobs[i].pid, NULL, 0);

    return 0;
}

```

OUTPUT:

```

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week9
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ nano jobs.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ gcc jobs.c -o jobs
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ ./jobs
Three background jobs created.

Active Jobs:
ID    PID    STATE
[1]    524    Running
[2]    525    Running
[3]    526    Running

After status update:

Active Jobs:
ID    PID    STATE
[1]    524    Completed
[2]    525    Completed
[3]    526    Running
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ |

```

OBSERVATION:

The program successfully implemented a job-listing mechanism using a job table. Multiple background processes were created using `fork()`, and their job IDs, PIDs, and states were stored and displayed. The `waitpid()` function with `WNOHANG` was used to check the status of each job without blocking. After the processes completed, their state

Task - 2 :

To design how to identify target job, transfer terminal control, resume processes, wait for completion, update job state, and test foreground switching.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        printf("Child: Job started. PID = %d\n", getpid());

        sleep(3);

        printf("Child: Job completed.\n");

        return 0;
    }

    setpgid(pid, pid);

    printf("Parent: Job created.\n");
    printf("Job PID: %d\n", pid);

    sleep(1);

    printf("Parent: Bringing job to foreground...\n");

    /*
     * Ignore SIGTTOU so the parent can safely
     * change terminal control.
     */
    signal(SIGTTOU, SIG_IGN);
```

```

/*
 * Ignore SIGTTOU so the parent can safely
 * change terminal control.
 */
signal(SIGTTOU, SIG_IGN);

/* Give terminal control to child process group */
tcsetpgrp(STDIN_FILENO, pid);

/* Resume the job */
kill(-pid, SIGCONT);

/* Wait for the foreground job */
waitpid(pid, &status, 0);

/* Take terminal control back */
tcsetpgrp(STDIN_FILENO, getpgrp());

if (WIFEXITED(status))
    printf("Parent: Foreground job completed.\n");

return 0;
}

```

OUTPUT:

```

ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ nano foreground.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ gcc foreground.c -o foreground
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ ./foreground
Parent: Job created.
Job PID: 653
Child: Job started. PID = 653
Parent: Bringing job to foreground...
Child: Job completed.
Parent: Foreground job completed.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ █

```

OBSERVATION:

The program successfully demonstrated foreground job switching and terminal control. A child process was created and identified using its PID, after which terminal control was transferred to the child process group. The process was resumed using SIGCONT, and waitpid() was used to wait for its completion. After the child terminated, terminal control was returned to the parent process, confirming successful foreground execution and job-state management.

SESSION 18:

Task - 1:

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handler(int sig)
{
    printf("Signal %d received by PID %d\n", sig, getpid());
}

int main()
{
    pid_t pid;

    signal(SIGUSR1, handler);

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());

        printf("Child waiting for signal...\n");
        pause();

        printf("Child resumed after receiving signal.\n");
        return 0;
    }

    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    sleep(2);

    printf("Parent sending SIGUSR1 to child...\n");
    kill(pid, SIGUSR1);

    waitpid(pid, NULL, 0);

    printf("Child process completed.\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week9
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ nano signals.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ gcc signals.c -o signals
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ ./signals
Child PID: 839
Parent PID: 838
Child waiting for signal...
Child PID: 839
Parent sending SIGUSR1 to child...
Signal 10 received by PID 839
Child resumed after receiving signal.
Child process completed.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ █
```

OBSERVATION:

The program successfully demonstrated signal delivery and handling between a parent and child process. The child process was created using `fork()` and remained suspended using `pause()` until it received a signal. The parent process used `kill()` to send `SIGUSR1` to the child, and the registered signal handler successfully detected and processed the signal. After receiving the signal, the child resumed execution and completed normally. This experiment demonstrated the basic working of signals, signal handlers, process synchronization, and inter-process communication.

Task - 2:

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

struct Job
{
    int id;
    pid_t pid;
    char status[20];
};

int main()
{
    struct Job job;
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child: Job started. PID = %d\n", getpid());

        while (1)
        {
            printf("Child: Job is running...\n");
            sleep(1);
        }

        job.id = 1;
        job.pid = pid;
        sprintf(job.status, "Running");

        printf("Parent: Job created.\n");
        printf("Job ID: %d\n", job.id);
        printf("PID: %d\n", job.pid);
        printf("Status: %s\n\n", job.status);

        sleep(2);
    }
}
```

GNU nano 8.7.1

```
}

job.id = 1;
job.pid = pid;
sprintf(job.status, "Running");

printf("Parent: Job created.\n");
printf("Job ID: %d\n", job.id);
printf("PID: %d\n", job.pid);
printf("Status: %s\n\n", job.status);

sleep(2);

printf("Parent: Stopping job...\n");
kill(pid, SIGSTOP);

sleep(1);

waitpid(pid, &status, WUNTRACED);

if (WIFSTOPPED(status))
{
    sprintf(job.status, "Stopped");
    printf("Job status: %s\n\n", job.status);
}

printf("Parent: Sending SIGCONT...\n");
kill(pid, SIGCONT);

sprintf(job.status, "Running");
printf("Job status: %s\n\n", job.status);

sleep(3);

printf("Parent: Terminating job...\n");
kill(pid, SIGTERM);

waitpid(pid, &status, 0);

sprintf(job.status, "Completed");

printf("Job status: %s\n", job.status);
printf("Background job recovery completed successfully.\n");

return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week9
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ nano resume_job.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ gcc resume_job.c -o resume_job
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$ ./resume_job
Parent: Job created.
Job ID: 1
PID: 1039
Status: Running

Child: Job started. PID = 1039
Child: Job is running...
Child: Job is running...
Parent: Stopping job...
Job status: Stopped

Parent: Sending SIGCONT...
Job status: Running

Child: Job is running...
Child: Job is running...
Child: Job is running...
Parent: Terminating job...
Job status: Completed
Background job recovery completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week9$
```

OBSERVATION:

The program successfully demonstrated background job recovery and status management. A child process was created and maintained in a job table with its Job ID, PID, and current status. The process was initially in the Running state and was then stopped using SIGSTOP. Its stopped state was verified using waitpid() and WUNTRACED. The process was successfully resumed by sending SIGCONT, and its status was updated back to Running. After verifying continued execution, the job was terminated and its status was updated to Completed. Thus, stopping, resuming, monitoring, and recovering a background job were successfully demonstrated.